

Übung zur Vorlesung Grundlagen Betriebssysteme und Rechnernetze

4. Übungsblatt, Abgabe bis Mo, 15. Dezember 2025, 10:00 Uhr

Hinweise zur Bearbeitung und Abgabe der Übung:

- Die Übungsaufgaben und Zusatzmaterial finden Sie auf Moodle:
<https://moodle2.uni-potsdam.de/course/view.php?id=47479>
- Abgabe in 3er-Gruppen; tragen Sie sich im Moodle-Kurs in eine Abgabegruppe ein! Auf allen Abgaben Namen und Matrikelnummern der Gruppenmitglieder nicht vergessen!
- Laden Sie Ihre Lösungen der Theorieaufgaben als PDF in Moodle hoch.
- Nutzen Sie für die Praxisaufgaben Git.UP! Klonen Sie sich für die jeweilige Aufgabe die Vorlage aus der Gruppe **GBR Vorlagen**. Sollte es für die Aufgabe keine Vorlage geben, zB: Aufgabe 1.1, erstellen Sie ein eigenes Repository. Achten Sie darauf ihre Git-Projekte im GBR Namespace zu erstellen. Für weitere Details folgen sie dem Manual auf Moodle.
- Halten Sie sich an die in den praktischen Aufgaben gegebenen Dateinamen und Methodennamen!
- Bitte keine Umlaute und Leerzeichen in Verzeichnissen und Dateinamen in der Abgabe verwenden! Ersetzen Sie Umlaute in Dateinamen (ä wird zu ae etc.)!
- Bei Problemen mit den Praxisaufgaben oder den Abgaben in GitLab legen Sie bitte in Ihrem GitLab-Repository ein Issue an und markieren darin einen Lehrenden! Bei Fragen zu den Aufgaben benutzen Sie bitte das Diskussionsforum in Moodle!
- Bitte kommentieren Sie Ihre Programme sinnvoll! Verwenden Sie sprechende Bezeichner und behandeln Sie mögliche Fehler!
- Stellen Sie sicher, dass Ihre Programme auf `tiree.lab.cs.uni-potsdam.de` compilierbar und lauffähig sind!

Gesamtpunktzahl des Aufgabenblattes: 20 Punkte.

Aufgabe 4.1: Syscall

Systemaufrufe werden üblicherweise als Softwareinterrupt realisiert. Die aufzurufende Behandlungsroutine wird durch eine eindeutige Nummer in der System Call Table identifiziert.

Bis jetzt haben Sie alle Systemaufrufe mithilfe von Methoden aus der C-Bibliothek aufgerufen. Jedoch existieren nicht für alle Systemaufrufe passende Bibliotheksfunktionen. In diesem Fall kann man Systemaufrufe selber mit der Methode `syscall` ausführen. Diese abstrahiert den platformspezifischen Assemblercode zum Auslösen eines Syscalls und stellt die CPU-Register nach Beendigung des Systemaufrufs wieder her.

- Um die Laufzeitunterschiede zwischen der Benutzung der C-Bibliothek und der Benutzung des Systemaufrufs mittels `syscall` am Beispiel von `gettimeofday` zu vergleichen, schreiben Sie zwei C-Programme, die jeweils den entsprechenden Aufruf enthalten. Wiederholen Sie den entsprechenden Aufruf in einer Schleife 10.000.000 Mal um einen messbaren Unterschied festzustellen.
- Messen Sie mit dem Kommandozeilentool `time` die Laufzeit Ihrer beiden Programme. Was stellen Sie fest? Erklären Sie die Zeitunterschiede!
- Verifizieren Sie ihre Vermutung mit Hilfe des Debuggers `gdb`. Geben Sie Screenshots der relevanten Stellen in Ihrem Debugger, die Ihre Vermutung untermauern, mit ab.

Hinweis: Die `syscall` number für `gettimeofday` ist als Makro `SYS_gettimeofday` in `sys/syscall.h` definiert.

(2+1+1 Punkte)

Aufgabe 4.2: Synchronisation gemeinsamer Variablen

Betrachten Sie das folgende Programm.

```
int k=3;
int* p;

void t1_f1(void){
    int x=5;
    p=&x;
    sleep(1);
}

void t1_f2(void){
    int y=7;
    k++;
    sleep(1);
}
```

```

void* t1_main(void* args){
    t1_f1();
    t1_f2();
    return NULL;
}

void* t2_main(void* args){
    sleep(1);
    k=k* *p;
    printf("%d_\n", k);
    return NULL;
}

int main(int argc, char ** argv){
    pthread_t threads[2];
    pthread_create(threads+1, NULL, t2_main, NULL);
    pthread_create(threads, NULL, t1_main, NULL);
    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);
    exit(0);
}

```

- Geben Sie mindestens 6 mögliche Ausgaben des Programms an und wie diese zustande kommen.
- Wieso kann die Belegung und die Benutzung der globalen Variable p wie in diesem Programm undefiniertes Verhalten hervorrufen?

Hinweis: Bei der Abgabe in Moodle finden Sie eine Version des Programms, welche den kommentierten Assemblercode enthält.

(2+1 Punkte)

Aufgabe 4.3: Synchronisation nebenläufiger Prozesse

Gegeben sei die folgende Semaphorlösung für das Readers&Writers-Problem:

Reader	Writer
(...)	(...)
start_read();	start_write();
read(Daten);	write(Daten);
end_read();	end_write();
(...)	(...)

Initialisierung gemeinsamer Variablen:

```
readcount = 0; /* Zaehler fuer Lesewuensche */
```

```

writecount = 0; /* Zaehler fuer Schreibwuensche */
mutex_rc = 1; /* binaeres Semaphor fuer exklusiven Zugriff auf readcount */
mutex_wc = 1; /* binaeres Semaphor fuer exklusiven Zugriff auf writecount */
mutex_str = 1; /* binaeres Semaphor fuer exklusiven Zugriff auf start_read */

sem_w = 1; /* binaeres Semaphor fuer exklusives Schreiben */
sem_r = 1; /* falls r=1 dann Lesen erlaubt, falls r=0 dann Lesen nicht erlaubt
           (d. h. Schreibwunsch liegt vor) */

start_read()
1  DOWN(mutex_str)
2  DOWN(sem_r);
3  DOWN(mutex_rc);
4  readcount++;
5  if (readcount==1) then DOWN(sem_w);
6  UP(mutex_rc);
7  UP(sem_r);
8  UP(mutex_str)

end_read()
1  DOWN(mutex_rc);
2  readcount--;
3  if (readcount==0) then UP(sem_w);
4  UP(mutex_rc);

start_write()
1  DOWN(mutex_wc);
2  writecount++;
3  if (writecount==1) then DOWN(sem_r);
4  UP(mutex_wc);
5  DOWN(sem_w);

end_write()
1  UP(sem_w);
2  DOWN(mutex_wc);
3  writecount--;
4  if (writecount==0) then UP(sem_r);
5  UP(mutex_wc);

```

Im Folgenden sei vorausgesetzt, dass jeder Prozess maximal drei Anweisungen (Zeilen) ausführen kann, bevor seine Zeitscheibe abläuft. Dabei sollen Funktionsaufrufe vereinfacht durch ihren Quelltext ersetzt werden (falls vorhanden). Das Schedulingverfahren sei ein Round-Robin-Verfahren. Dokumentieren Sie den Ablauf tabellarisch unter Angabe der veränderten gemeinsamen Variablen, wenn

- a) ein Prozess R_1 liest für 2 Zeitscheiben, während der zweiten Zeitscheibe trifft noch ein Leseprozess R_2 und danach noch innerhalb derselben Zeitscheibe ein Schreibprozess W_1 ein!

- b) ein Prozess W_1 schreibt für 1 Zeitscheiben, während der ersten Zeitscheibe trifft ein Leseprozess R_1 und während der zweiten Zeitscheibe trifft ein weiterer Schreibprozess W_2 ein!

Hinweis: Orientieren Sie sich an der Darstellung für das Beispiel aus der Übung!

(2 + 2 Punkte)

Aufgabe 4.4: Shared Memory und Semaphore

In der Vorlesung wurde Ihnen das Platzbuchungssystem „Arche Noah“ vorgestellt. Erstellen Sie ein Platzbuchungssystem für ein beliebiges Verkehrsmittel mit n Plätzen. In diesem System existieren ein Masterprozess und beliebig viele Workerprozesse.

Die Workerprozesse füllen eine gemeinsame Buchungsliste (Array) mit zufälligen Kundennummern, indem sie eine zufällige Nummer generieren, verifizieren, dass diese noch nicht in der Buchungsliste vorhanden ist, und anschließend auf den nächsten freien Platz eintragen. Sobald die Buchungsliste voll ist, soll der Master diese ausgeben.

Der Masterprozess soll dabei nicht aktiv warten (busy-waiting), bis die Buchungsliste voll ist. Benutzen Sie Semaphore! Des Weiteren soll es möglich sein, die Prozesse in einer beliebigen Reihenfolge zu starten.

- a) Implementieren Sie die beiden Programme `master.c` und `client.c` unabhängig von einander in C! Es wird genau ein Masterprozess gestartet. Vom Workerprogramm können mehrere Instanzen gestartet werden. Benutzen Sie POSIX-Shared-Memory (`man shm_overview`) und POSIX-Semaphore (`man sem_overview`). Die Größe n des Shared-Memories soll in beiden Programmen als symbolische Konstante festgelegt sein.
- b) Semaphor-Operationen können durch Signale unterbrochen werden. Achten Sie darauf, dass eine kritische Sektion nicht betreten wird, weil der Semaphor-Aufruf durch einen Signalhandler unterbrochen wurde. Erweitern Sie Ihre Programme aus a) so, dass dieser Fehlerfall nicht eintritt!

(5 + 1 Punkte)

Aufgabe 4.5: Das Proc-Dateisystem

Das Proc-Dateisystem ist ein virtuelles Dateisystem, welches Nutzern die Möglichkeit bietet, mit dem Kernel zu interagieren. Nicht-root-Nutzer können keine Veränderungen vornehmen, jedoch immer noch viele Informationen lesen. Unter `/proc/cpuinfo` und `/proc/meminfo` findet man so z.B. Informationen zur CPU und über den Hauptspeicher.

Unter `/proc/PID/` können Informationen über den Prozess mit der PID angezeigt werden.

Das Programm `pfiletest.c` öffnet eine Datei und führt `pthread_create` einmal aus. Nach 3 Minuten wird die Datei geschlossen und nach 3 weiteren Minuten wird das Programm beendet. `pfiletest.c` liegt im Material-Ordner zur Lehrveranstaltung.

- a) Führen Sie `pfiletest` wie folgt aus: `./pfiletest 2> /tmp/err_uid < /dev/null`. Dabei ersetzen Sie `uid` durch eine beliebige Nummer, z.B. Ihre UID. Unter `/proc/PID/fd/` werden die geöffneten Dateideskriptoren angezeigt. Welche Dateideskriptoren sind das für den Programmaufruf? Welcher Dateideskriptor verbirgt sich hinter welcher Zahl und worauf verweisen diese?
- b) Unter `/proc/PID/task/TID/fd` kann man die Dateideskriptoren pro Thread angezeigt bekommen. Unterscheiden sich diese während der Ausführung des Programms? Betrachten Sie das Programm erneut nach dem zweiten `wait` auf `barrier`, falls nicht ein Thread, sondern ein Kindprozess erzeugt worden wäre. Begründen Sie!

(2+1 Punkte)